

Quasi-Newton Methods: BFGS and L-BFGS

Chapter 11 of Course Notes

Dr. Robert Hildebrand

Grado Department of Industrial & Systems Engineering
Virginia Tech

Conjugate Directions

Linear Independence of Conjugate Directions

Conjugate Direction Method for Quadratic Programs

Properties of the Conjugate Direction Method

Exercises

Gradient-based optimization methods:

	Steepest Descent	Quasi-Newton (BFGS)	Newton
Cost/iter	$O(n)$	$O(n^2)$	$O(n^3)$
Convergence	Linear	Superlinear	Quadratic
Requires	∇f	∇f only	$\nabla f, \nabla^2 f$
Storage	$O(n)$	$O(n^2)$	$O(n^2)$

The central question: Can we get Newton-like convergence without computing the Hessian?

✓ **Answer:** Yes! Quasi-Newton methods build an approximate Hessian from gradient differences.

Problem Statement and Scope

Unconstrained minimization:

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x})$$

Assumptions:

- f is twice continuously differentiable
- Gradients $\nabla f(\mathbf{x})$ are available (not Hessians)
- For large n , storing an $n \times n$ matrix is expensive

Lecture outline:

1. Motivation: Why Quasi-Newton?
2. The secant condition
3. Deriving BFGS
4. BFGS properties and convergence
5. The inverse update formula

Conjugate Directions

Linear Independence of Conjugate Directions

Conjugate Direction Method for Quadratic Programs

Properties of the Conjugate Direction Method

Exercises

The Cost of Newton's Method

Newton's update: $\mathbf{x}_{k+1} = \mathbf{x}_k - [\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k)$

Why great:

- **Quadratic convergence** near $\mathbf{x}^*$
- Converges in 1 step for quadratic functions
- Uses curvature information (scale-invariant)

Why impractical:

1. **Computing** $\nabla^2 f(\mathbf{x}_k)$ requires $\frac{n(n+1)}{2}$ second derivatives
For $n = 1000$: $\sim 500,000$ entries per iteration!
2. **Solving** the linear system $\nabla^2 f(\mathbf{x}_k) \mathbf{p}_k = -\nabla f(\mathbf{x}_k)$ costs $O(n^3)$ per iteration
3. **Not available** analytically for many real problems (e.g., ML, simulations)

Quasi-Newton Idea: Approximate the Hessian

Key insight: We don't need the *exact* Hessian, just a good *approximation* that we can update cheaply.

Newton iteration: $\mathbf{x}_{k+1} = \mathbf{x}_k - \mathbf{B}_k^{-1} \nabla f(\mathbf{x}_k)$

where $\mathbf{B}_k \approx \nabla^2 f(\mathbf{x}_k)$

Quasi-Newton requirements:

1. $\mathbf{B}_k$ must capture useful **curvature information** from gradients
2. Update from $\mathbf{B}_k$ to $\mathbf{B}_{k+1}$ must be **cheap** (rank-1 or rank-2)
3. $\mathbf{B}_k$ must remain **positive definite** $\Rightarrow$ descent directions
4. Still achieve **superlinear convergence**

The answer: BFGS method satisfies all four conditions!

Conjugate Directions

Linear Independence of Conjugate Directions

Conjugate Direction Method for Quadratic Programs

Properties of the Conjugate Direction Method

Exercises

Extracting Curvature from Gradients

By the mean value theorem:

$$\nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k) \approx \nabla^2 f(\mathbf{x}_k)(\mathbf{x}_{k+1} - \mathbf{x}_k)$$

Define:

$$\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k \quad (\text{step taken})$$

$$\mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k) \quad (\text{gradient change})$$

The secant equation:

$$\mathbf{B}_{k+1} \mathbf{s}_k = \mathbf{y}_k$$

Meaning: The new Hessian approximation $\mathbf{B}_{k+1}$ acts like the true Hessian along the most recent step direction.

In 1D analogy: The secant condition is the slope of the secant line through two gradient evaluations:

$$B = \frac{y_k}{s_k} = \frac{f'(x_{k+1}) - f'(x_k)}{x_{k+1} - x_k}$$

Why the Secant Condition Isn't Enough

Underdetermined system: The secant condition $\mathbf{B}_{k+1}\mathbf{s}_k = \mathbf{y}_k$ gives n equations.

A symmetric $n \times n$ matrix has $\frac{n(n+1)}{2}$ independent entries.

For $n = 100$: 100 equations, 5050 unknowns — massively underdetermined!

Additional requirements:

1. **Symmetry:** $\mathbf{B}_{k+1} = \mathbf{B}_{k+1}^\top$ (like the true Hessian)
2. **Positive definiteness:** $\mathbf{B}_{k+1} \succ 0$ (ensures descent)
3. **Closeness to $\mathbf{B}_k$:** The update $\Delta_k = \mathbf{B}_{k+1} - \mathbf{B}_k$ should be low-rank
(Preserve curvature already learned)

Minimal-rank options:

- Rank 1 update: SR1 (Symmetric Rank-One) — may lose positive definiteness
- Rank 2 update: BFGS — preserves positive definiteness

Conjugate Directions

Linear Independence of Conjugate Directions

Conjugate Direction Method for Quadratic Programs

Properties of the Conjugate Direction Method

Exercises

BFGS: Closest Matrix to $\mathbf{B}_k$

Mathematical formulation:

$$\begin{array}{ll} \min_{\mathbf{B}} & \|\mathbf{B} - \mathbf{B}_k\|_W \\ \text{s.t.} & \mathbf{B} = \mathbf{B}^\top, \quad \mathbf{B} \mathbf{s}_k = \mathbf{y}_k \end{array}$$

where $\|\cdot\|_W$ is a weighted Frobenius norm designed to be scale-invariant.

Intuition: Among all symmetric matrices satisfying the secant condition, pick the one that changes $\mathbf{B}_k$ as little as possible (preserve accumulated curvature).

The BFGS rank-2 update formula:

$$\mathbf{B}_{k+1} = \mathbf{B}_k + \frac{\mathbf{y}_k \mathbf{y}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k} - \frac{\mathbf{B}_k \mathbf{s}_k \mathbf{s}_k^\top \mathbf{B}_k}{\mathbf{s}_k^\top \mathbf{B}_k \mathbf{s}_k}$$

Structure:

- First term: adds information from gradient change $\mathbf{y}_k$
- Second term: removes old information along direction $\mathbf{s}_k$

Why Rank-2? Sketch of Derivation

Ansatz: Assume $\mathbf{B}_{k+1} = \mathbf{B}_k + \alpha \mathbf{u} \mathbf{u}^\top + \beta \mathbf{v} \mathbf{v}^\top$ (rank-2 symmetric update)

Step 1: Apply secant condition:

$$\begin{aligned}(\mathbf{B}_k + \alpha \mathbf{u} \mathbf{u}^\top + \beta \mathbf{v} \mathbf{v}^\top) \mathbf{s}_k &= \mathbf{y}_k \\ \mathbf{B}_k \mathbf{s}_k + \alpha (\mathbf{u}^\top \mathbf{s}_k) \mathbf{u} + \beta (\mathbf{v}^\top \mathbf{s}_k) \mathbf{v} &= \mathbf{y}_k\end{aligned}$$

Step 2: Choose $\mathbf{u}$ and $\mathbf{v}$ proportional to terms in the update:

$$\begin{aligned}\mathbf{u} &\propto (\mathbf{y}_k - \mathbf{B}_k \mathbf{s}_k) \quad (\text{the "prediction error"}) \\ \mathbf{v} &\propto \mathbf{B}_k \mathbf{s}_k \quad (\text{the previous Hessian component})\end{aligned}$$

Step 3: Solve for α and β using the secant and symmetry constraints

Result: The rank-2 BFGS formula with specific normalizations.

(Full derivation in Nocedal & Wright Ch. 6)

Key Property: Positive Definiteness Preservation

Theorem

BFGS Preserves Positive Definiteness If $\mathbf{B}_k \succ 0$ (positive definite) and $\mathbf{y}_k^\top \mathbf{s}_k > 0$, then $\mathbf{B}_{k+1} \succ 0$.

Proof sketch: The BFGS rank-2 update can be written as:

$$\mathbf{B}_{k+1} = \mathbf{B}_k + \frac{1}{\mathbf{y}_k^\top \mathbf{s}_k} [(\mathbf{y}_k - \mathbf{B}_k \mathbf{s}_k)(\mathbf{y}_k - \mathbf{B}_k \mathbf{s}_k)^\top] - \frac{\mathbf{B}_k \mathbf{s}_k \mathbf{s}_k^\top \mathbf{B}_k}{\mathbf{s}_k^\top \mathbf{B}_k \mathbf{s}_k}$$

Both rank-1 terms have the form $\mathbf{v}\mathbf{v}^\top$ (positive semidefinite) or corrections that preserve PD.

Curvature condition: $\mathbf{y}_k^\top \mathbf{s}_k > 0$ is guaranteed by [Wolfe line search](#):

Wolfe curvature condition:

$$\nabla f(\mathbf{x}_{k+1})^\top \mathbf{s}_k \geq c_2 \nabla f(\mathbf{x}_k)^\top \mathbf{s}_k$$

with $0 < c_2 < 1$. This implies $\mathbf{y}_k^\top \mathbf{s}_k > 0$ for descent directions.

Self-Correcting Property

Even if BFGS fails to find the true Hessian:

BFGS has a remarkable property: errors correct themselves over iterations.

Key fact: If all iterates are in a region where f is strongly convex, and we use Wolfe line search, then:

- The sequence of BFGS approximations $\mathbf{B}_1, \mathbf{B}_2, \dots$ converges to $\nabla^2 f(\mathbf{x}^*)$
- Even if early approximations are poor, the method self-corrects
- This is why BFGS is so robust in practice

Contrast with SR1:

- SR1 is not guaranteed to preserve positive definiteness
- In practice, SR1 sometimes produces indefinite approximations
- But when it works, SR1 can have fewer iterations because it's "more general"
- Trade-off: robustness vs. flexibility

Conjugate Directions

Linear Independence of Conjugate Directions

Conjugate Direction Method for Quadratic Programs

Properties of the Conjugate Direction Method

Exercises

Why Update the Inverse?

Naive approach: Update $\mathbf{B}_k$ and solve $\mathbf{B}_{k+1}\mathbf{p}_k = -\nabla f(\mathbf{x}_k)$ each iteration

But solving an $n \times n$ linear system costs $O(n^3)$ by Gaussian elimination.

Smart approach: Update $\mathbf{H}_k \approx [\nabla^2 f(\mathbf{x}_k)]^{-1}$ directly

$$\mathbf{p}_k = -\mathbf{H}_k \nabla f(\mathbf{x}_k) \Rightarrow O(n^2) \text{ matrix-vector product}$$

This reduces per-iteration cost from $O(n^3)$ to $O(n^2)$.

How to derive the inverse update?

Use the [Sherman-Morrison formula](#) (or direct matrix manipulation).

Sherman-Morrison Formula

For invertible $\mathbf{A}$ and vectors $\mathbf{u}, \mathbf{v}$:

$$(\mathbf{A} + \mathbf{u}\mathbf{v}^\top)^{-1} = \mathbf{A}^{-1} - \frac{\mathbf{A}^{-1}\mathbf{u}\mathbf{v}^\top\mathbf{A}^{-1}}{1 + \mathbf{v}^\top\mathbf{A}^{-1}\mathbf{u}}$$

(provided $1 + \mathbf{v}^\top\mathbf{A}^{-1}\mathbf{u} \neq 0$)

BFGS inverse update derivation:

Starting from the direct formula (which has rank-2 structure), apply Sherman-Morrison twice to obtain:

$$\mathbf{H}_{k+1} = \left(\mathbf{I} - \frac{\mathbf{s}_k \mathbf{y}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k} \right) \mathbf{H}_k \left(\mathbf{I} - \frac{\mathbf{y}_k \mathbf{s}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k} \right) + \frac{\mathbf{s}_k \mathbf{s}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k}$$

where $\rho_k = \frac{1}{\mathbf{y}_k^\top \mathbf{s}_k}$ simplifies this to:

$$\mathbf{H}_{k+1} = (\mathbf{I} - \rho_k \mathbf{s}_k \mathbf{y}_k^\top) \mathbf{H}_k (\mathbf{I} - \rho_k \mathbf{y}_k \mathbf{s}_k^\top) + \rho_k \mathbf{s}_k \mathbf{s}_k^\top$$

Interpretation: Rank-2 update that modifies the inverse directly without solving a linear system.

Forward vs. Inverse Update: Worked Example

Consider $f(\mathbf{x}) = x_1^2 + 4x_2^2$ with $\mathbf{x}_0 = (2, 1)^\top$. Initial: $\mathbf{H}_0 = \mathbf{I}$.

Step 0: $\nabla f(\mathbf{x}_0) = (4, 8)^\top$, $\mathbf{p}_0 = -\mathbf{H}_0 \nabla f(\mathbf{x}_0) = (-4, -8)^\top$

Line search $\Rightarrow \alpha_0 = 0.1 \Rightarrow \mathbf{x}_1 = (1.6, 0.2)^\top$

At iteration 1:

$\mathbf{s}_0 = (-0.4, -0.8)^\top$, $\mathbf{y}_0 = \nabla f(\mathbf{x}_1) - \nabla f(\mathbf{x}_0) = (3.2, -6.4)^\top - (4, 8)^\top = (-0.8, -14.4)^\top$

$$\rho_0 = \frac{1}{(-0.8) \cdot (-0.4) + (-14.4) \cdot (-0.8)} = \frac{1}{0.32 + 11.52} = \frac{1}{11.84}$$

Inverse update: Apply the formula above to get $\mathbf{H}_1$

Key advantage: We compute $\mathbf{p}_1 = -\mathbf{H}_1 \nabla f(\mathbf{x}_1)$ without solving $\mathbf{B}_1 \mathbf{p}_1 = -\nabla f(\mathbf{x}_1)$

Saves $O(n^3)$ Gaussian elimination per iteration!

Conjugate Directions

Linear Independence of Conjugate Directions

Conjugate Direction Method for Quadratic Programs

Properties of the Conjugate Direction Method

Exercises

Complete BFGS Algorithm

Input: $\mathbf{x}_0$, tolerance ϵ , $m_{\max}$

Initialize: $\mathbf{H}_0 = \mathbf{I}$ (or scaled version)

for $k = 0, 1, 2, \dots$ **do**

$\nabla_k \leftarrow \nabla f(\mathbf{x}_k)$

if $\|\nabla_k\| < \epsilon$ **then** *STOP*

$\mathbf{p}_k \leftarrow -\mathbf{H}_k \nabla_k$ *% Newton-like step*

$\alpha_k \leftarrow \text{LineSearch}(\mathbf{x}_k, \mathbf{p}_k)$ *% Wolfe conditions*

$\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \alpha_k \mathbf{p}_k$

$\mathbf{s}_k \leftarrow \alpha_k \mathbf{p}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$

$\mathbf{y}_k \leftarrow \nabla f(\mathbf{x}_{k+1}) - \nabla_k$

if $\mathbf{y}_k^\top \mathbf{s}_k > 0$ **then**

$\rho_k \leftarrow 1/(\mathbf{y}_k^\top \mathbf{s}_k)$

$\mathbf{H}_{k+1} \leftarrow (\mathbf{I} - \rho_k \mathbf{s}_k \mathbf{y}_k^\top) \mathbf{H}_k (\mathbf{I} - \rho_k \mathbf{y}_k \mathbf{s}_k^\top) + \rho_k \mathbf{s}_k \mathbf{s}_k^\top$

else

$\mathbf{H}_{k+1} \leftarrow \mathbf{H}_k$ *% skip update (rare with Wolfe)*

end if

end for

Convergence Theory

Theorem

Global and Superlinear Convergence of BFGS Suppose f is twice continuously differentiable and strongly convex on an open set containing $\mathbf{x}^*$. Under Wolfe line search and with $\mathbf{H}_0 = \mathbf{I}$, BFGS satisfies:

1. **Global convergence:** $\mathbf{x}_k \rightarrow \mathbf{x}^*$ as $k \rightarrow \infty$
2. **Superlinear convergence:**

$$\lim_{k \rightarrow \infty} \frac{\|\mathbf{x}_{k+1} - \mathbf{x}^*\|}{\|\mathbf{x}_k - \mathbf{x}^*\|} = 0$$

Why superlinear, not quadratic?

- BFGS approximates the Hessian but doesn't use it exactly
- As we approach $\mathbf{x}^*$, the approximation improves
- Convergence rate improves dynamically (superlinear = between linear and quadratic)

Dennis-Moré condition: A sufficient condition for superlinearity is

$$\lim_{k \rightarrow \infty} \frac{\|(\mathbf{B}_{k+1} - \nabla^2 f(\mathbf{x}^*))\mathbf{s}_k\|}{\|\mathbf{s}_k\|} = 0$$

i.e., BFGS becomes “more Newton-like” as $k \rightarrow \infty$.

Conjugate Directions

Linear Independence of Conjugate Directions

Conjugate Direction Method for Quadratic Programs

Properties of the Conjugate Direction Method

Exercises

Example: Minimizing $f(\mathbf{x}) = x_1^2 + 4x_2^2$

True Hessian: $\nabla^2 f = \text{diag}(2, 8)$, inverse: $[\nabla^2 f]^{-1} = \text{diag}(1/2, 1/8)$

Starting point: $\mathbf{x}_0 = (1, 1)^\top$, $\nabla_0 = (2, 8)^\top$, $\mathbf{H}_0 = \mathbf{I}$

k	$\mathbf{x}_k$	$\ \nabla_k\ $	$\mathbf{H}_k$ (approx)	Reduction in f
0	(1, 1)	8.25	$\mathbf{I}$	(start)
1	(0.33, 0.21)	0.96	rank-2 corrected	85%
2	(0.003, 0.005)	0.07	closer to true inv.	99.5%
3	$\approx (0, 0)$	$< 10^{-6}$	converged	(machine eps)

Key observations:

- After 1 iteration: BFGS has already captured significant curvature
- After 2 iterations: nearly converged (superlinear rate)
- The $\mathbf{H}_k$ sequence adapts toward the true inverse Hessian

Conjugate Directions

Linear Independence of Conjugate Directions

Conjugate Direction Method for Quadratic Programs

Properties of the Conjugate Direction Method

Exercises

The Memory Problem: Why L-BFGS?

BFGS bottleneck for large n :

- Store $\mathbf{H}_k$: $n \times n$ dense matrix = $O(n^2)$ memory
- Compute $\mathbf{H}_k \nabla_k$: $O(n^2)$ operations
- For $n = 10^6$ (neural networks, PINNs): ~ 8 TB storage!

Observation: The BFGS update is recursive:

$$\mathbf{H}_{k+1} = f(\mathbf{H}_k, \mathbf{s}_k, \mathbf{y}_k)$$

We can *implicitly* represent $\mathbf{H}_k$ by unrolling the last m update steps:

$$\mathbf{H}_k = f(\mathbf{H}_{k-m}^0, \{\mathbf{s}_i, \mathbf{y}_i\}_{i=k-m}^{k-1})$$

L-BFGS idea:

- Store only the last m pairs $(\mathbf{s}_i, \mathbf{y}_i)$ for $i = k - m, \dots, k - 1$
- Store a simple initial matrix $\mathbf{H}_k^0$ (usually $\gamma_k \mathbf{I}$)
- Compute $\mathbf{H}_k \nabla_k$ on-the-fly using the two-loop recursion

Storage: $O(mn)$ instead of $O(n^2)$ **Cost:** $O(mn)$ instead of $O(n^2)$

The Two-Loop Recursion Algorithm

Input: Gradient ∇_k , memory pairs $\{(\mathbf{s}_i, \mathbf{y}_i)\}_{i=k-m}^{k-1}$, initial scaling γ_k

$\mathbf{q} \leftarrow \nabla_k$

Loop 1 (right to left):

for $i = k-1, k-2, \dots, k-m$ **do**

$\rho_i \leftarrow 1/(\mathbf{y}_i^\top \mathbf{s}_i)$

$\alpha_i \leftarrow \rho_i \mathbf{s}_i^\top \mathbf{q}$ *% store for Loop 2*

$\mathbf{q} \leftarrow \mathbf{q} - \alpha_i \mathbf{y}_i$

end for

$\mathbf{r} \leftarrow \gamma_k \mathbf{q}$ *% apply initial Hessian*

Loop 2 (left to right):

for $i = k-m, k-m+1, \dots, k-1$ **do**

$\rho_i \leftarrow 1/(\mathbf{y}_i^\top \mathbf{s}_i)$

$\beta \leftarrow \rho_i \mathbf{y}_i^\top \mathbf{r}$

$\mathbf{r} \leftarrow \mathbf{r} + (\alpha_i - \beta) \mathbf{s}_i$

end for

Output: $\mathbf{r} = \mathbf{H}_k \nabla_k$ (the direction $\mathbf{p}_k = -\mathbf{r}$)

Cost: $4mn$ flops per iteration (linear in n for fixed m !)

Why the Two-Loop Recursion Works

The BFGS update in unrolled form:

Let $V_i = \mathbf{I} - \rho_i \mathbf{y}_i \mathbf{s}_i^\top$. Then:

$$\mathbf{H}_k = (V_{k-1}^\top \cdots V_{k-m}^\top) \mathbf{H}_k^0 (V_{k-m} \cdots V_{k-1}) + \text{rank-1 correction terms}$$

Loop 1 applies the right-multiplications $V_{k-1}, \dots, V_{k-m}$ to ∇_k :

$$\mathbf{q} \leftarrow (V_{k-1} \cdots V_{k-m}) \nabla_k$$

Stores intermediate coefficients α_i for later use.

Middle step: Multiply by initial approximation: $\mathbf{r} = \mathbf{H}_k^0 \mathbf{q} = \gamma_k \mathbf{q}$

Loop 2 applies left-multiplications $V_{k-m}^\top, \dots, V_{k-1}^\top$ and adds rank-1 corrections:

$$\mathbf{r} \leftarrow (V_{k-m}^\top \cdots V_{k-1}^\top) \mathbf{r} + \text{corrections}$$

Result: $\mathbf{r} = \mathbf{H}_k \nabla_k$ without ever forming the matrix!

Example: Two-Loop Recursion with $m = 2$

Consider $f(\mathbf{x}) = x_1^2 + 2x_2^2$, at iteration $k = 4$ with $m = 2$ stored pairs:

$$\mathbf{s}_2 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \mathbf{y}_2 = \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \quad \rho_2 = 1/2$$

$$\mathbf{s}_3 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad \mathbf{y}_3 = \begin{pmatrix} 0 \\ 4 \end{pmatrix}, \quad \rho_3 = 1/4$$

Current gradient: $\nabla_4 = (6, 8)^\top$

Initial scaling: $\gamma_4 = \frac{\mathbf{s}_3^\top \mathbf{y}_3}{\mathbf{y}_3^\top \mathbf{y}_3} = 1/4$

Loop 1: $i = 3$: $\alpha_3 = \rho_3(0, 1)^\top (6, 8) = 1/4 \cdot 8 = 2$,

$\mathbf{q} \leftarrow (6, 8)^\top - 2(0, 4)^\top = (6, 0)^\top$

$i = 2$: $\alpha_2 = \rho_2(1, 0)^\top (6, 0) = 1/2 \cdot 6 = 3$,

$\mathbf{q} \leftarrow (6, 0)^\top - 3(2, 0)^\top = (0, 0)^\top$

Middle: $\mathbf{r} = 1/4 \cdot (0, 0)^\top = (0, 0)^\top$

Loop 2: $i = 2$: $\beta = 1/2 \cdot (2, 0)^\top (0, 0) = 0$,

$\mathbf{r} \leftarrow (0, 0)^\top + (3 - 0)(1, 0)^\top = (3, 0)^\top$

$i = 3$: $\beta = 0$, $\mathbf{r} \leftarrow (3, 0)^\top + (2 - 0)(0, 1)^\top = (3, 2)^\top$ ✓

Initial Hessian Scaling and Memory Parameter

Scaling γ_k : Instead of $\mathbf{H}_k^0 = \mathbf{I}$, use:

$$\gamma_k = \frac{\mathbf{s}_{k-1}^\top \mathbf{y}_{k-1}}{\mathbf{y}_{k-1}^\top \mathbf{y}_{k-1}}$$

Intuition: Adapts the scale based on recent curvature.

Memory parameter m :

- $m = 1$: minimal memory, faster per iteration, slower convergence
- $m = 5-20$: typical (good balance)
- $m = n$: BFGS (full memory)

Storage/cost for fixed n :

	Steepest Descent	L-BFGS	BFGS
Storage	$O(n)$	$O(mn)$	$O(n^2)$
Cost/iter	$O(n)$	$O(mn)$	$O(n^2)$
Convergence	Linear	Superlinear	Superlinear

For large-scale ML: L-BFGS is the [workhorse method](#).

Conjugate Directions

Linear Independence of Conjugate Directions

Conjugate Direction Method for Quadratic Programs

Properties of the Conjugate Direction Method

Exercises

Methods at a Glance

	SD	CG	BFGS	Newton
Cost/iter	$O(n)$	$O(n)$	$O(n^2)$	$O(n^3)$
Convergence	Linear	Linear*	Super	Quadratic
Requires	∇f	∇f	∇f	$\nabla f, \nabla^2 f$
Storage	$O(n)$	$O(n)$	$O(n^2)$	$O(n^2)$
Robustness	High	Medium	High	Medium

* CG on quadratics: n iterations; general f : similar to SD convergence

When to use BFGS:

- Medium-scale problems ($100 < n < 10000$)
- Need faster convergence than CG or SD
- Can store $n \times n$ dense matrix
- Hessian not available

When to use L-BFGS:

- Large-scale problems ($n > 10000$)
- Need memory-efficient quasi-Newton method
- Machine learning, inverse problems, PINNs
- Standard in industry (TensorFlow, PyTorch, scipy)

Real-World Application: PINNs

Physics-Informed Neural Networks (PINNs) solve PDEs by minimizing loss:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{data}} + \lambda \mathcal{L}_{\text{PDE}}$$

where θ are network weights (10^4 to 10^7 parameters).

Challenge: Gradient descent is slow (high condition number), Newton is impossible ($O(n^3)$).

Solution: Use L-BFGS with $m = 100$ – 1000

- Captures curvature without storing full Hessian
- Superlinear convergence vs. linear for SGD
- Typical speedup: 5–50x vs. Adam optimizer

Example result: Burgers' equation PDE

- Adam: 50K iterations, relative error $\sim 1\%$
- L-BFGS: 5K iterations, relative error $\sim 0.01\%$

Conjugate Directions

Linear Independence of Conjugate Directions

Conjugate Direction Method for Quadratic Programs

Properties of the Conjugate Direction Method

Exercises

Exercise 1: One BFGS Iteration by Hand

Exercise 1

Consider $f(x_1, x_2) = (x_1 - 1)^2 + 4(x_2 - 2)^2$.

Starting at $\mathbf{x}_0 = (2, 3)^\top$ with $\mathbf{H}_0 = \mathbf{I}$, perform one BFGS iteration with exact line search.

1. Compute ∇_0 and the direction $\mathbf{p}_0 = -\mathbf{H}_0 \nabla_0$
2. Use exact line search (set derivative to zero) to find α_0^* and $\mathbf{x}_1$
3. Compute $\mathbf{s}_0, \mathbf{y}_0, \rho_0$
4. Apply the BFGS inverse update to compute $\mathbf{H}_1$
5. Verify that $\mathbf{H}_1 \succ 0$ (check eigenvalues if desired)

Exercise 2: Two-Loop Recursion

Exercise 2

Implement the two-loop recursion algorithm for $m = 2$ memory pairs with:

$$\mathbf{s}_0 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \mathbf{y}_0 = \begin{pmatrix} 1 \\ 0.5 \\ 0 \end{pmatrix}$$

$$\mathbf{s}_1 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \quad \mathbf{y}_1 = \begin{pmatrix} 0.2 \\ 1 \\ 0.3 \end{pmatrix}$$

$$\nabla_k = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, \quad \gamma_k = 1$$

Step through Loop 1, the middle step, and Loop 2 to compute $\mathbf{H}_k \nabla_k$.

Summary: Key Takeaways

BFGS and L-BFGS at a Glance:

1. **Quasi-Newton idea:** Approximate $\nabla^2 f$ using gradient differences via secant condition
2. **BFGS:** Rank-2 symmetric update that preserves positive definiteness and achieves superlinear convergence
3. **Inverse update formula:** Directly update $\mathbf{H}_k = [\nabla^2 f]^{-1}$ to avoid solving linear systems
4. **L-BFGS:** Store only last m pairs and use two-loop recursion to compute search directions in $O(mn)$ time
5. **Practical impact:**
 - BFGS: for medium-scale problems
 - L-BFGS: for large-scale (ML, PDEs, inverse problems)
6. **Advantages:** No Hessian needed, only gradients; superlinear; robust with Wolfe line search

Next lecture: Trust region methods (Lecture 15)

Key ideas:

- Alternative to line search: define a region around $\mathbf{x}_k$ where we trust a quadratic model
- Can use any Hessian approximation (including BFGS) within a trust region
- Often more robust than line search when f is nonsmooth or ill-behaved near $\mathbf{x}_k$

Further reading:

- Nocedal & Wright, *Numerical Optimization*, Chapters 6–7
- Byrd, Nocedal, Schnabel (1994): L-BFGS paper
- Curtis & Nocedal (2020): Quasi-Newton methods survey